



MemoRec: a recommender system for assisting modelers in specifying metamodels

Juri Di Rocco¹ · Davide Di Ruscio¹ · Claudio Di Sipio¹ · Phuong T. Nguyen¹ · Alfonso Pierantonio¹

Received: 5 November 2020 / Revised: 2 November 2021 / Accepted: 10 February 2022 / Published online: 29 March 2022
© The Author(s) 2022, Corrected publication 2022

Abstract

Model-driven engineering has been widely applied in software development, aiming to facilitate the coordination among various stakeholders. Such a methodology allows for a more efficient and effective development process. Nevertheless, modeling is a strenuous activity that requires proper knowledge of components, attributes, and logic to reach the level of abstraction required by the application domain. In particular, metamodels play an important role in several paradigms, and specifying wrong entities or attributes in metamodels can negatively impact on the quality of the produced artifacts as well as other elements of the whole process. During the metamodeling phase, modelers can benefit from assistance to avoid mistakes, e.g., getting recommendations like metaclasses and structural features relevant to the metamodel being defined. However, suitable machinery is needed to mine data from repositories of existing modeling artifacts and compute recommendations. In this work, we propose MemoRec, a novel approach that makes use of a collaborative filtering strategy to recommend valuable entities related to the metamodel under construction. Our approach can provide suggestions related to both metaclasses and structured features that should be added in the metamodel under definition. We assess the quality of the work with respect to different metrics, i.e., success rate, precision, and recall. The results demonstrate that MemoRec is capable of suggesting relevant items given a partial metamodel and supporting modelers in their task.

Keywords Model-driven engineering · Recommender systems · Collaborative filtering techniques

1 Introduction

Model-driven engineering (MDE) [37] development relies heavily on metamodels and models which are used to represent an abstraction of real-world entities as well as to produce application code automatically. As a result, the modeling activity represents the core of this paradigm and

should be carefully addressed to avoid possible errors in application deployment. Nowadays, modelers are equipped with several tools that support their tasks with different features, i.e., graphical environment, drag-and-drop utilities, and auto-completion. So far, different modeling assistants have been proposed to support modelers in their daily activities [2,11,23,24]. Nevertheless, most of them deal with testing or repairing [1], and to the best of our knowledge there have been no approaches dedicated to supporting metamodel specification. In particular, packages and metaclasses are the building blocks of a metamodel, but so far there exists no tool to help modelers effectively specify these artifacts. In fact, while working on a metamodel, modelers might expect to get recommendations consisting of relevant metaclasses or structural features that can be further integrated. However, due to a huge amount of available resources, searching for suitable artifacts is a daunting task. Under the circumstances, we see an urgent need for suitable machinery to mine data from open source platforms such as GitHub. Among others, we are interested in finding which packages and metaclasses

Communicated by Joerg Kienzle.

✉ Davide Di Ruscio
davide.diruscio@univaq.it

Juri Di Rocco
juri.dirocco@univaq.it

Claudio Di Sipio
claudio.disipio@graduate.univaq.it

Phuong T. Nguyen
phuong.nguyen@univaq.it

Alfonso Pierantonio
alfonso.pierantonio@univaq.it

¹ Università degli studi dell'Aquila, L'Aquila, Italy

can be added to the metamodel under development, given that other packages and metaclasses are already defined.

In this work, we aim to provide modelers with an automated assistant, providing support during metamodeling activities. We propose MemoRec, a recommender system that exploits a context-aware collaborative filtering technique [4] to recommend relevant artifacts related to the modeling domain. In particular, MemoRec has been conceptualized by learning from a series of recommender systems developed in the scope of the CROSSMINER project [7] to mine open source software, providing developers with various artifacts, including topics [5], API invocations, and source code [30]. MemoRec goes one step further to assist metamodeling activities by processing input data with four different encoding techniques (i.e., different selections of what information has to be kept from a metamodel). More importantly, we tailor the internal design to compute similarity among metamodels in an efficient way. Given a metamodel partially specified by the modeler, MemoRec is able to suggest two types of artifacts, namely (i) metaclasses at the level of package; and (ii) structural features for a given metaclass. To properly capture the current context, we rely on the data encoding technique that has been successfully conceptualized in our previous work [29]. To our best knowledge, this is the first attempt to support MDE application development, exploiting collaborative filtering techniques. Thus, the contributions of our work are summarized as follows.

- A recommender system, named MemoRec, to provide modelers with classes and structural features relevant to the metamodels under development;
- An empirical evaluation of the conceived system on two real metamodel datasets, employing a set of well-defined metrics commonly used in the recommender systems domain, i.e., success rate, precision, recall, and F_1 score;
- The replication package of the tool has been made available to facilitate future research.¹

Through a careful observation, we realized that there exist no other tools that perform the same tasks, and thus it is not possible to compare our tool with any baseline. In the scope of this paper, we evaluate the performance of our proposed tool by relying on the k-fold cross-validation technique [18], aiming to investigate its practicality in real-world settings.

The paper is structured into the following sections. Section 2 shows a motivating example as well as background related to the context-aware collaborative filtering technique. In Sect. 3, we present MemoRec and its main components. The evaluation is presented in Sect. 4, while the results are analyzed in Sect. 5. A qualitative discussion of MemoRec is given in Sect. 6. Section 7 presents the work that is related

to the approach presented in this paper. We conclude and discuss possible future work in Sect. 8.

2 Motivations and background

This section describes the issues we tackle in this work and provides an overview of the background. In particular, Sect. 2.1 gives a motivating example where recommendations for completing a metamodel are needed. Afterward, Sect. 2.2 presents the context-aware collaborative-filtering technique and its application in recommending API function calls and usage pattern as a base for further presentations.

2.1 Motivating example

We present a motivating example to illustrate the need for proper recommendations. The considered context is a modeler who is developing the Web metamodel depicted in Fig. 1a. The metamodel consists of two packages, i.e., *Web* and *Data*, where the former models the presentation concepts and contains three classes, i.e., *Static* and *Dynamic* that inherit from *Page*. The latter groups data concepts such as *Entity* and *Field*. By considering a metamodel and an active context as input, a model assistant is expected to recommend both additional classes and structural features that the metamodel under development should incorporate. An *active context* is a package or a class that the modeler is attempting to define when asking for recommendations. In Fig. 1, let us suppose that the modeler is asking for additional classes and structural features for two given active contexts, i.e., the *Web* package and the *Page* class, being marked with the blue and the red color, respectively.

Figure 1b depicts two possible recommendations for the given context, i.e., *css* structural feature (marked in red) within the *Page* class and *Form* class (marked in blue) within the *Web* package. To be concrete, the *Form* class allows the modeler to define complete HTML resources. By considering the possible structural feature recommendations *Page* class, the usage of *css* adds common and interesting information to *Pages* in the *Web* domain. In other words, a modeler assistant should consider the active context to predict suitable recommendations that help the modeler complete the context.

2.2 Context-aware collaborative filtering technique

A context-aware collaborative filtering recommender system provides recommendations to users items that have been bought by similar users in similar contexts [35,36]. Based on this premise, we successfully developed a recommender system named FOCUS to provide developers with API function calls and usage patterns [28]. We modeled the mutual

¹ <https://github.com/MDEGroup/MemoRec>.

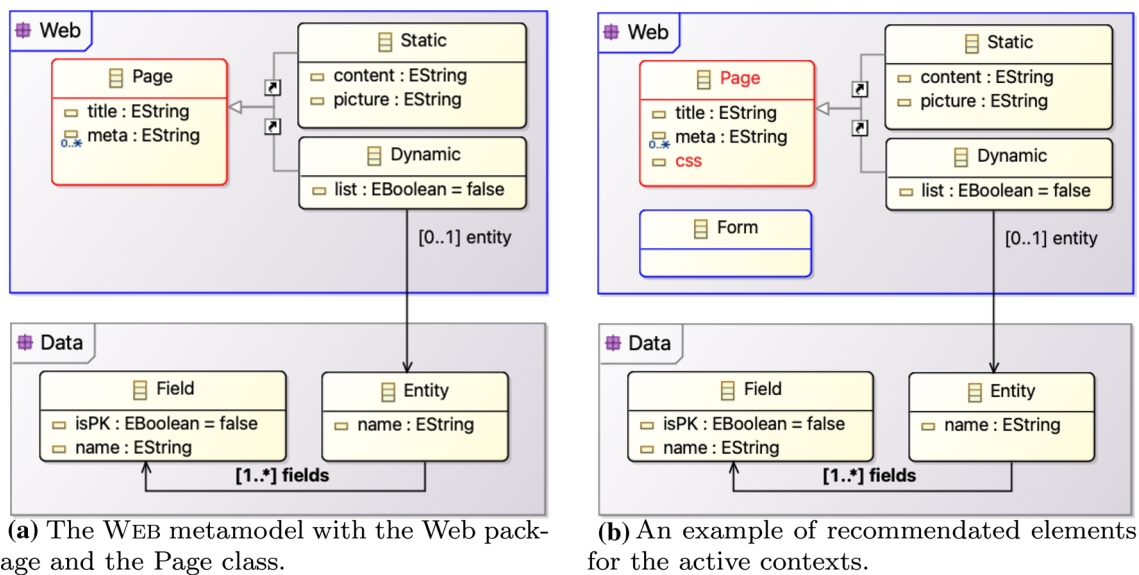


Fig. 1 **a** The WEB metamodel with the Web package and the Page class. **b** An example of recommended elements for the active contexts. The explanatory WEB metamodel

relationships among projects using a tensor and mined API usage from the most similar projects.

The successful deployment of different variants of collaborative filtering techniques allows us to transfer the acquired knowledge into the MDE domain. We build MemoRec by redesigning and customizing FOCUS to support the completion of metamodels. Mining from similar objects is a building block of collaborative filtering techniques, and we exploit this to provide recommendations for metamodels under development. In particular, our approach works based on the assumption that: “if metamodels share some common artifacts, then they should probably have other common artifacts.” In this respect, MemoRec mines metaclasses and structural features from similar metamodels, given an input metamodel. The following section presents our conceived approach to recommend useful elements for a metamodel being under development.

3 Proposed approach

MemoRec provides modelers with recommendations, which can be helpful while defining a metamodel, by considering the existing portion of the metamodel as active context. The system makes use of a graph representation to encode the relationships among various metamodels artifacts and generates recommendations employing a context-aware collaborative filtering technique [36]. In addition, we exploit a tailored textual representation of metamodels, which are encoded to enable the extraction of the containing knowledge to feed as input for the recommendation engine.

The architecture of MemoRec is depicted in Fig. 2. To provide recommendations, MemoRec accepts as input a set of *Metamodel Repositories* ①. Afterward, the *Metamodel Encoder* component ② extracts packages and classes pairs as well as class and structural feature pairs from the metamodel being developed. *Metamodel Comparator*, a subcomponent of *Similarity Calculator* ③, measures the similarity between the metamodels stored in the repositories and the metamodel under specification. Using the set of metamodels and the information extracted by *Metamodel Encoder*, the *Data Encoder* component ④ computes rating matrices. Given an active context of a metamodel, i.e., package or class, *Item Comparator* computes the similarities between packages and classes. From the similarity scores, *Recommendation Engine* ⑤ generates recommendations, either as a ranked list of classes if the active context is a package, or a ranked list of structural features if the active context is a class. In the remainder of this section, we present in greater details each of these components.

3.1 Metamodel encoder

A metamodel defines the abstract concepts of a domain where concepts, as well as the relationships among them, are expressed by the used modeling infrastructure. In particular, a metamodel consists of *Packages* that aggregate similar concepts expressed by *Classes*. *Classes* consist of structural features, i.e., attributes and references. Moreover, *Classes* can inherit structural features from other classes.

Being inspired by our recent work [29], we employ four encoding schemes to represent different views concerning the terms extracted from packages, classes, and structural

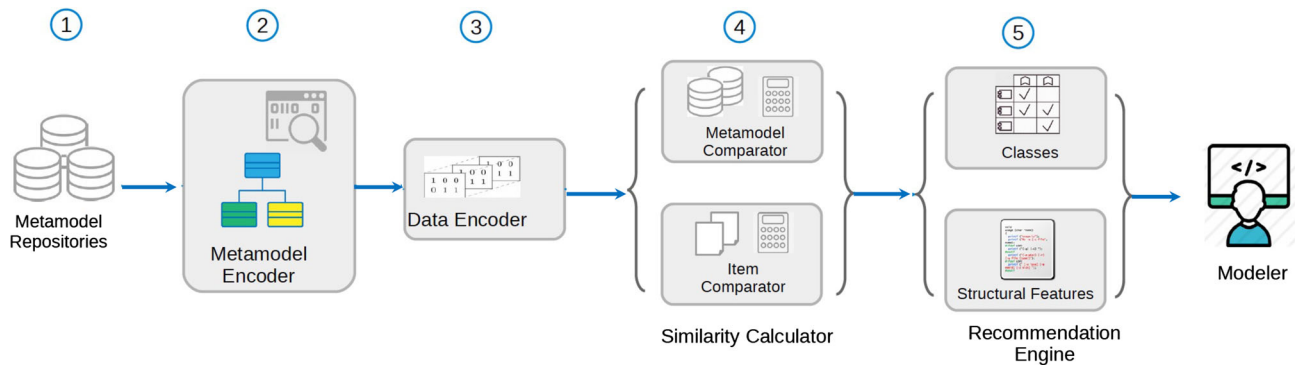


Fig. 2 Overview of MemoRec's architecture

features named instance. Each scheme has been used to elicit relevant information from the input metamodels according to different granularity levels. In particular, we make use of two *standard encoding* (SE) scheme for recommending structural features within a context class and two other *improved encoding* (IE) scheme for supporting classes within a package context. In particular, we consider the following definitions:

- SE_s : it includes pairs in the form of $\langle \text{class_name} \rangle \# \langle \text{structural_feature_name} \rangle$ for each structural feature contained within a class. This encoding scheme is used to suggest additional structural features within a given class context;
- IE_s : it consists of pairs $\langle \text{class_name} \rangle \# \langle \text{structural_feature_name} \rangle$ for each structural feature contained within a class. Moreover, it includes structural features inherited from the super classes. In our previous work [8], we studied how structural features are used with hierarchies. On the one hand, we found out that increasing the number of metaclasses with super-types decreases the average number of structural features directly specified in a metaclass, since structural features are spread through the class hierarchies. On the other hand, the average number of structural features including the inherited ones is uncorrelated with the number of metaclasses with super-types. We anticipate that, by including the inherited structural features in IE_s , we will be able to increase the informative part of the encoding. We use this scheme to suggest additional structural features within a given class context;
- SE_c : it includes pairs $\langle \text{package_name} \rangle \# \langle \text{class_name} \rangle$ for each class contained within a package. This encoding scheme is utilized in providing additional classes within a given package context;
- IE_c : it flattens packages and classes and encodes classes within a default artificial package. We use this encoding scheme to suggest additional classes within a given package context. Since metamodels consist of few *ePackages* [8], we envision that a flatten representation of

classes, i.e., by bypassing the Package/Class containment can help MemoRec to consider more metamodels and to extract classes from different top similar metamodels.

An encoding scheme depends on two main factors:

- the purpose of the recommendation; depending on the type of the recommended items (e.g., structural features, classes, specialization/generalization of a metaclass, etc.), the encoding scheme should be tailored to support the identified recommendation goal;
- the prediction performance; the encoding scheme strongly impacts on the prediction performance. For this reason, the identification of suitable encoding scheme is an iterative process where the encodings are incrementally improved to maximize the prediction performance for a specific purpose.

As future work, further encoding schemes could be provided to target different kinds of recommendations. For instance, an encoding scheme representing the inheritance relations between classes could suggest a possible set of generalizations or specializations for a given metaclass in the active context. In addition, a different encoding scheme could be used to include types for the recommended structural features.

Figure 3 depicts an extract of the four encoding schemes related to the metamodels depicted in Fig. 1a. In the following subsection, we show how the pairs *package/class* and *class/structural feature* relationships are encoded. In Sect. 4, we evaluated MemoRec by considering the four encoding schemes described in this section, i.e., SE_s , IE_s , SE_c , and IE_c .

3.2 Data encoder

Once package and class pairs as well as class and structural features have been extracted, MemoRec represents the relationships among them using two rating matrices to sup-

1 Page#title	1 Page#title
2 Page#meta	2 Page#meta
3 Static#Content	3 Static#Content
4 Static#picture	4 Static#picture
5 Dynamic#list	5 Static#title
6 Dynamic#entity	6 Static#meta
7 Entity#name	7 Dynamic#list
8 Entity#fields	8 Dynamic#entity
9 Field#isPK	9 Dynamic#title
10 Field#name	10 Dynamic#meta
(a) SE_s encoding	(b) IE_s encoding
1 Web#Page	1 Package#Page
2 Web#Static	2 Package#Static
3 Web#Dynamic	3 Package#Dynamic
4 Data#Entity	4 Package#Entity
5 Data#Field	5 Package#Field
(c) SE_c encoding	(d) IE_c encoding

Fig. 3 The Web metamodel data extraction

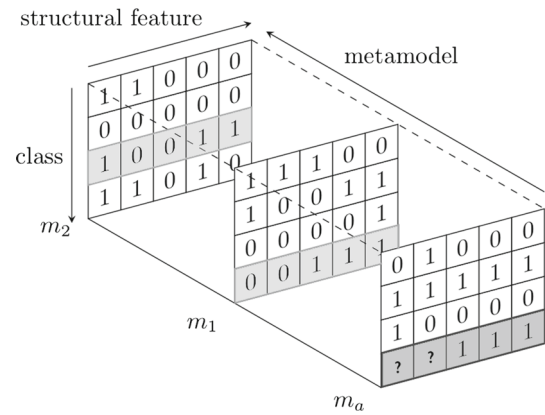
Table 1 Package-class feature rating matrix combined with SE_c for the Web metamodel

	Page	Static	Dynamic	Entity	Field
Web	1	1	1	0	0
Data	0	0	0	1	1

port class and structural feature recommendations. Given a metamodel, each row in the matrix corresponds to a package (class), and each column represents a class (structural feature). A cell is set to 1 if the package (class) in the corresponding row contains the class (structural feature) in the column, otherwise it is set to 0.

Tables 1 and 2 illustrate how the metamodel depicted in Fig. 1a is encoded into corresponding rating matrices. In particular, Table 1 shows the rating matrix combined with SE_c , whereas Table 2 reports the rating matrix combined with IE_s .

A 3D context-based ratings matrix is introduced to model the intrinsic relationships among various metamodels, package (classes) and class (structural feature). The third dimension of this matrix represents a metamodel, which is analogous to the so-called context in context-aware collaborative filtering systems. For example, Fig. 4 depicts three metamodels $M = (m_a, m_1, m_2)$ represented by three slices with four classes and five structural features: m_a is the *active metamodel* and it has an *active context* highlighted in dark gray. Both m_1 and m_2 are complete metamodels similar to m_a , and they are called *background data*, as they serve as a base for the recommendation process. On the one hand, the more background metamodel we have, the better is the chance that we recommend relevant structural features. On the other hand, increasing the number of top similar metamodels will enlarge the ratings matrix and thus will add more computational complexity.

**Fig. 4** Matrix representation of metamodels w.r.t. structural features and classes

3.3 Similarity calculator

The recommendation of suitable metamodel items, i.e., classes or structural features, is derived from similar metamodels and the active context, i.e., packages or classes. *Similarity Calculator* is a generic component, it can be used to compute similarity for both classes and structural features. Given an active context of a metamodel under development, it is essential to find the subset of the most similar ones, and then the most similar contexts in that set of metamodels. Based on the active context type, we create a weighted directed graph that models the relationships among metamodels and structural features to compute similarities. Moreover, we implemented a graph-based similarity function [6,28] to calculate the similarities among metamodels.

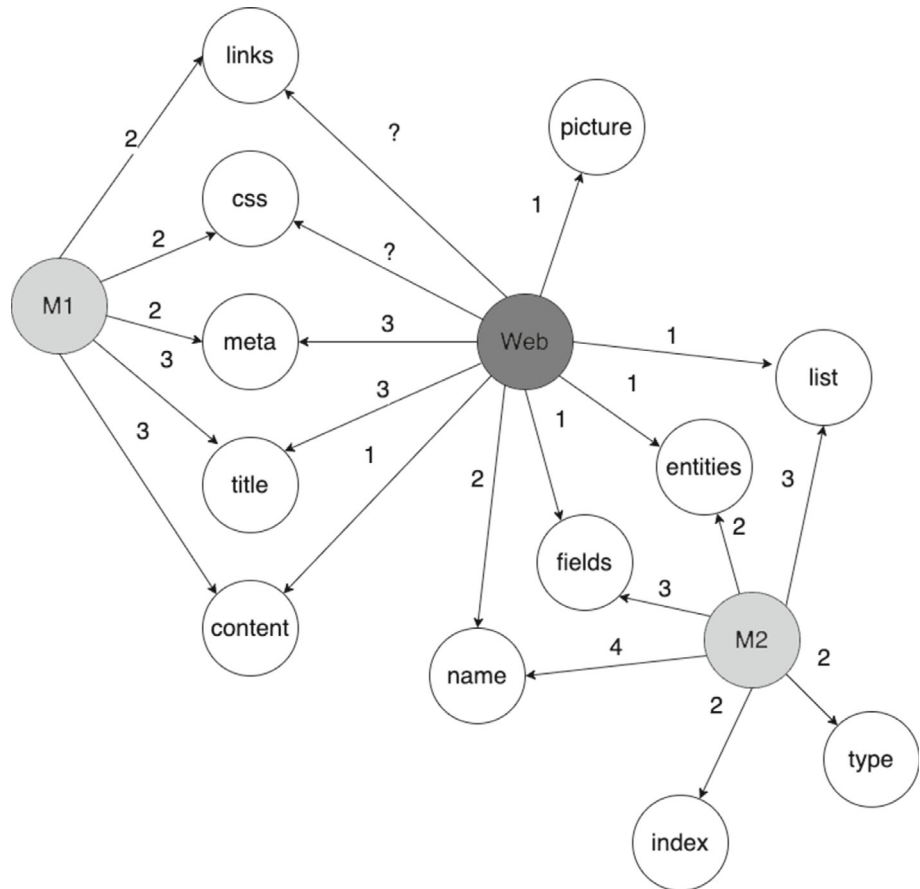
In particular, we used two graph representations to support both class and structural feature recommendations. Each node in the graph represents either a metamodel or a structural feature. If metamodel m contains structural feature f , then there is a directed edge from m to f . The weight of the edge $m \rightarrow f$ corresponds to the number of times m includes f . Figure 5 depicts the graph for the set of projects in Fig. 4: White nodes represent structural features, blue nodes represent most similar metamodels to the input ones depicted in green. For instance, the *Web* metamodel has five classes and two of them define the attribute *name*. As a result, the edge *Web* $\rightarrow$ *name* contains a weight of 2. In the graph, a question mark represents missing information, i.e., for the active declaration in *Web*, we need to find out if invocations *links* and *css* shall be included or not.

Given the node representing a metamodel m , there are nodes connected to m via different edges, and they are called neighbor nodes. Considering $(n_1, n_2, \dots, n_l)$ as a set of neighbor nodes of m , the feature set of m is the vector $\phi = (\phi_1, \phi_2, \dots, \phi_l)$, where ϕ_k is the weight of node n_k , and computed using the *term-frequency inverse document fre-*

Table 2 Class-structural feature rating matrix combined with IE_s for the Web metamodel

	Title	Meta	Content	Picture	List	Entity	Name	Fields	isPK
Page	1	1	0	0	0	0	0	0	0
Static	1	1	1	1	0	0	0	0	0
Dynamic	1	1	0	0	1	1	0	0	0
Entity	0	0	0	0	0	0	1	1	0
Field	0	0	0	0	0	0	1	0	1

Fig. 5 Graph representation of metamodels and structural features



quency function computed by the following formula:

$$\phi_k = f_{n_k} * \log \left(\frac{|M|}{a_{n_k}} \right) \quad (1)$$

where f_{n_k} is the weight of the edge $m \rightarrow n_k$; $|M|$ is the number of all considered metamodels; and a_{n_k} is the number of metamodels connected to n_k .

The similarity between two metamodels m and n is comprehended as the cosine between their feature vectors $\phi = \{\phi_k\}_{k=1,\dots,l}$ and $\omega = \{\omega_j\}_{j=1,\dots,z}$, computed below:

$$\text{sim}_1(m, n) = \frac{\sum_{t=1}^{\pi} \phi_t \times \omega_t}{\sqrt{\sum_{t=1}^{\pi} (\phi_t)^2} \times \sqrt{\sum_{t=1}^{\pi} (\omega_t)^2}} \quad (2)$$

where π is the cardinality of the union of the sets of nodes by m and n .

Finally, the similarity between classes c and d is calculated with the Jaccard index given below:

$$\text{sim}_2(c, d) = \frac{|\mathbb{F}(c) \cap \mathbb{F}(d)|}{|\mathbb{F}(c) \cup \mathbb{F}(d)|} \quad (3)$$

where $\mathbb{F}(c)$ and $\mathbb{F}(d)$ are the sets of structural features for c and d , respectively.

By referring to the motivation example proposed in Sect. 2.1 and depicted in Fig. 5, we present a concrete application of the proposed formalisms. Table 3 reports the ϕ vectors for the *Web*, *M1*, and *M2* metamodels. Then, Table 4 lists the cosine similarity among vectors. Each cell reports the sim_1 score between the metamodels represented in the corre-

Table 3 ϕ vectors for the metamodels depicted in Fig. 5

	Links	css	Media	Title	Content	Picture	Name	Fields	Entities	List	Index	Type
Web	0	0	0.528	0.528	0	0	0	-0.301	-0.301	-0.301	0	0
M1	0	0	0	0.528	0.528	0	0	0	0	0	0	0
M2	0	0	0	0	0	0	1.204	0.528	0	0.528	0.602	0.602

Table 4 sim_1 matrix for the metamodels depicted in Fig. 5

	Web	M1	M2
Web	1	0.41	-0.21
M1	0.41	1	0
M2	-0.21	0	1

sponding column and row. According to the results reported in Table 4, the *Web* metamodel is more similar to *M1* than to *M2*.

3.4 Recommendation engine

This component is used to generate a ranked list of relevant items, i.e., classes and structural features that depend on the metamodel context, i.e., package and class. In the rest of this section, we present structural feature recommendations based on the class context. Analogously, we apply the same approach for recommending classes within packages.

Figure 4 depicts an instance of structural features rating matrices. In particular, the active metamodel m_a already includes three classes, and the modeler is working on the fourth class, corresponding to the last row of the matrix. The active class c_a contains two structural features, represented in the last two columns of the matrix, i.e., cells marked with 1. The first two cells are filled with a question mark (?), implying that at the time of consideration, it is not clear whether these two structural features should also be added into c_a . *Recommendation Engine* computes the missing ratings to predict additional structural features for the active class by exploiting the following collaborative filtering formula [4,28]:

$$r_{c,f,m} = \bar{r}_c + \frac{\sum_{d \in \text{topsim}(c)} (R_{d,f,m} - \bar{r}_d) \cdot \text{sim}_2(c, d)}{\sum_{d \in \text{topsim}(c)} \text{sim}_2(c, d)} \quad (4)$$

Equation (4) is used to compute a score for the cell representing structural feature f , class c of metamodel m , where $\text{topsim}(c)$ is the set of top- N similar classes of c , $\text{sim}_2(c, d)$ is the similarity between two classes c and d , computed by Eq. (3); $\bar{r}_c$ and $\bar{r}_d$ are calculated by averaging out all the ratings of c and d , respectively; $R_{d,f,m}$ is the combined rating of d for f in all the similar metamodels, computed in

Eq. (5) [4].

$$R_{d,f,m} = \frac{\sum_{n \in \text{topsim}(m)} r_{d,f,n} \cdot \text{sim}_1(m, n)}{\sum_{n \in \text{topsim}(m)} \text{sim}_1(m, n)} \quad (5)$$

where $\text{topsim}(m)$ is the set of top similar metamodels of m ; and $\text{sim}_1(m, n)$ is the similarity between metamodels m and n , calculated by means of Eq. (2). Equation (5) suggests that given the active metamodel, a more similar metamodel is assigned a higher weight. This makes sense in practice, since similar metamodels contain more relevant structural features than less similar metamodels. Using Eq. (4) we compute all the missing ratings in the active class and get a ranked list of structural features with real scores in descending order. The list is then provided to modelers as recommendations.

4 Evaluation

This section describes the datasets and the process we conceived to evaluate MemoRec. In particular, Sect. 4.1 presents the research questions to study the performance of our proposed approach. Section 4.2 gives an overview of the datasets used in the evaluation. The methodology and metrics are described in Sects. 4.3 and 4.4, respectively.

4.1 Research questions

The following research questions are considered to investigate MemoRec's recommendation performance:

- **RQ₁**: *How well can MemoRec provide recommendations with different configurations?* We examine different configurations of MemoRec, i.e., the number of recommended items as well as the number of neighbor metamodels, to find the settings that bring the best performance.
- **RQ₂**: *How does the training data affect the performance of MemoRec?* We study the outputs of MemoRec by considering two different datasets to assess to what extent their quality can have ripple effects on the prediction accuracy of MemoRec;
- **RQ₃**: *How do the encoding schemes affect the performance of MemoRec?* Since the definition of a suitable encoding scheme is an iterative process, we compared

Table 5 Datasets

Number of artifacts	D_1	D_2
Packages	669	3140
Metaclasses	17,840	62,214
Structural features	31,688	159,323
Attributes	14,436	86,273
References	17,252	73,050

refined versions of the encodings, i.e., IE_c and IE_s , with the initial ones, i.e., SE_c and SE_s to pin down which of them facilitates the best recommendation outcome for MemoRec.

4.2 Data extraction

To evaluate the proposed approach, we exploited two independent datasets namely D_1 and D_2 as shown in Table 5, and they are described below.

- D_1 is a **curated** dataset [26], which consists of 555 metamodels mined from GitHub and already labeled by humans. In particular, its metamodels have been already classified into nine categories, i.e., *Bibliography*, *Issue tracker*, *Project build*, *Review system*, *Database*, *Office tools*, *Petrinet*, *State machine*, and *Requirements specification*. Though MemoRec does not require the input data to be labeled, such predefined categories are beneficial to the recommendation as there is a high similarity among the metamodels within a category. This is important since MemoRec heavily relies on similarity to function (see Sect. 3), i.e., given a metamodel, it searches for relevant items from similar metamodels;
- D_2 is a **raw and randomly collected** dataset using the GitHub API [14]. To aim for a reliable evaluation of MemoRec, we identified and filtered out from the dataset all the duplicated metamodels, resulting in a final set with 2,151 metamodels. By means of the GitHub API [14] we searched for files with the `.ecore` extension, which corresponds to Ecore metamodels. Due to the restrictions imposed by GitHub, e.g., it returns a maximum of 1,000 elements per query, we had to perform the searches by iteratively varying the query keywords. In particular, we used the `extension` qualifier as a base to search for `ecore` files. Then, we refined the query by adding typical `ecore` keywords, i.e., `ePackage`, `xml`, `eClass`, to name a few. Afterward, all the discovered metamodels were downloaded and collected in a dedicated folder. We removed all the files that we cannot directly parse with the EMF facilities [40] from the corpus of collected artifacts. Finally, we removed the duplicated metamodels by

the following process: (i) a hash is computed for every collected `ecore` file based on its content; (ii) the obtained hashes are used to build a hashmap where the key is the hash itself, and the value is the corresponding file; (iii) if a duplicated key occurs in the map, we assume that the corresponding `ecore` is a duplicate and it is discarded.

4.3 Methodology

As described in Sect. 3, MemoRec can recommend classes or structural features, i.e., attributes and references, depending on the recommendation context, i.e., packages or classes. For this reason, we perform the experiments by exploiting both classes and structural feature recommendations. In the rest of this section, we use classes within packages and structural features within classes as the recommendation objective.

To study if MemoRec is applicable in real-world settings, we perform an offline evaluation by simulating the behavior of a modeler who partially defines a metamodel and needs practical recommendations on how to do next. Figure 6 depicts the evaluation process with three consecutive steps, i.e., *Data Preparation*, *Recommendation Production*, and *Outcome Evaluation* explained as follows.

- **Data preparation** As shown in Fig. 6, starting from an input *Dataset*, we split it into two independent parts, i.e., *Training data* and *Testing data* (*Split ten-fold*). The former corresponds to the metamodels collected ex-ante, whereas the latter represents the metamodel being modeled, or the active metamodel. The *tenfold* cross-validation technique is used to conduct the evaluation as follows. The dataset is split into ten equal parts, one part represents the testing set and the remaining nine parts are combined to create a training set. We consider a modeler who is defining a metamodel m , so some parts of m are removed to mimic an actual metamodeling task: Some packages/classes are already available in the active metamodel and the system should recommend additional packages/classes to be incorporated. For each metamodel in the *Testing data*, by the *Split input data* phase, a random package (class) that, together with the remaining packages/classes, is selected to be used as *Query data*. In the considered context, the first class/structural feature is kept as query data and all the others are taken out to be used as ground-truth (GT) *data*. In other words, by the *Split input data* phase, package (class) is selected as the active context c . For c , only the selected classes are provided as query, while the rest is removed and saved as ground-truth data.
- **Recommendation production** In this phase, the extracted *Query data* and *Training data* are fed as input for MemoRec, which in turn computes the final *Recommendations*. It is important to remark that the current version of Mem-

oRec can recommend classes and structural features. The types of the recommended attributes and relationships are not supported yet. This represents our next step to further develop the proposed approach.

- **Outcome evaluation** The performance of MemoRec is measured by comparing the recommendation outcomes with the ground-truth data (*GT data*), exploiting the quality metrics, i.e., *Success rate*, *Precision*, and *Recall* which are presented in Sect. 4.4.

4.4 Metrics

We introduce the following notations as a base for further presentation:

- N is the cutoff value for the ranked list of recommendations. MemoRec returns a ranked list of recommendation items, and those on the top of the list are considered to be more relevant to the given context. The cutoff N is used to select the top items. For instance, if MemoRec retrieves four items, e.g., *FormElement*, *Form*, *Link*, and *Page*, and the cutoff value N is set to 2, then only the first two recommended items, e.g., *FormElement* and *Form* are provided as the final recommendation;
- k corresponds to the number of *top-similar* neighbor metamodels MemoRec considers to predict suggested items;
- $REC_N(m)$ is the *top-N* recommended items for m ;
- $GT(m)$ is defined as the list of classes/structural features that are saved as ground-truth data for metamodel m ;
- $TP_N(m)$ is the set of true positives, i.e., items in the *top-N* list that match with those in the ground-truth data, $TP_N(m) = GT(m) \cap REC_N(m)$.

Then the metrics, i.e., success rate, precision, and recall, are defined as follows.

▷ **SR@N** Given the testing set of metamodels M , success rate measures the ratio of queries that have at least a matched item among the total number of queries.

$$SR@N = \frac{\text{count}_{m \in M}(|TP_N(m)| > 0)}{|M|} \quad (6)$$

In particular, SR stands for success rate, and @N corresponds to a cutoff value of N . For example, SR@2 means success rate for a recommended list with 2 items.

▷ **Precision, Recall, and F₁-score (F-measure)** These metrics are utilized to measure the *accuracy* of the recommendation results. In particular, *precision* corresponds to the ratio of $TP_N(m)$ among the number of recommended items, *recall* is the ratio of $TP_N(m)$ belonging to $GT(m)$, whereas

F_1 -score is the harmonic mean of precision and recall.

$$\text{Precision} = \frac{|TP_N(m)|}{N} \quad (7)$$

$$\text{Recall} = \frac{|TP_N(m)|}{|GT(m)|} \quad (8)$$

$$F_1 = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} \quad (9)$$

▷ **Recommendation time** To measure the time needed to perform a prediction, we used a laptop with 2.7 GHz Intel Core i7 quad-core 16GB RAM, and macOS Catalina 10.15.5.

5 Experimental results

This section analyzes the performance obtained by running MemoRec on the considered datasets. The three research questions are addressed in Sects. 5.1, 5.2, and 5.3. Section 5.4 discusses possible threats to validity.

5.1 RQ₁: How well can MemoRec provide recommendations with different configurations?

We conducted experiments on the curated dataset (i.e., D_1) by varying the number of neighbor nodes k of the input metamodel, i.e., $k = \{1, 5, 10, 15, 20\}$, and the value of N , i.e., $N = \{1, 10, 20\}$. The rationale behind the selection of those values is as follows. First, we should not present a long list of recommended items since it may confuse the modeler; thus, we select $N = 20$ as the maximum value. Second, since the number of neighborhood items impacts on the computational complexity (cf. Eq. 5), it is impractical to use a large number of metamodels as neighbors. Therefore, we consider the following values $k = \{1, 5, 10, 15, 20\}$. In the experiments, we use SE_c and SE_s as encoding schemes to compute recommendations for classes and structural features, respectively. (The results related to the adoption of the other encoding schemes are presented in the discussion of RQ₃.)

Tables 6 and 7 show the average success rates obtained by running the tenfold cross-validation technique to recommend structural features and classes, respectively, by using different cutoff values N . In particular, Table 6 depicts the success rate obtained for recommending structural features classes. According to Table 6, we can observe that using more neighbor metamodels to compute recommendations brings a better success rate when the first recommendation item is considered. For instance, MemoRec gets a success rate@1 of 0.153 and 0.202 when $k = 1$ and $k = 20$, respectively. This is not confirmed when a longer list of recommendation items is considered, i.e., $N = \{10, 20\}$. Moreover, MemoRec yields a better performance when we increase the cutoff value N . Take as an example, for $k = 20$, $N = 1$, the obtained success rate

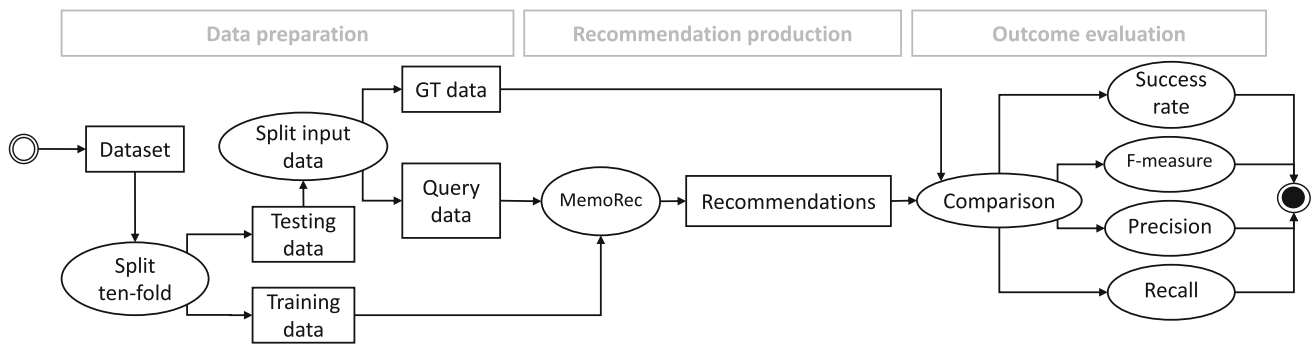


Fig. 6 Evaluation process

Table 6 Success rate for structural feature recommendations, $k = \{1, 5, 10, 15, 20\}$, by considering the $\mathbf{D}_1$ dataset

k	SR@1	SR@10	SR@20
1	0.152	0.394	0.501
5	0.181	0.406	0.488
10	0.202	0.419	0.502
15	0.200	0.392	0.494
20	0.202	0.362	0.479

is 0.202 which is less than a half of 0.479, the corresponding value when $N = 20$. A longer list of recommended items means an increase in the match rate; however, the modeler may tire of skimming through it. Thus, in practice, we should choose a suitable cutoff value N .

Table 7 reports the success rate obtained with class recommendations. Partly similar to the results presented for recommending structural features classes in Table 6, we see that incorporating more neighbors to compute recommendations is useful for a small k , i.e., $k = \{1, 5, 10\}$. However, starting from $k = 15$, there is a decrease in success rate by all the cutoff values N . We suppose that this happens due to the adoption of new neighbors that introduces only noise.

Answer to RQ₁ Considering a certain number of similar metamodells contributes to more relevant recommendations. Using data encoded with the SE_c and SE_s encoding schemes allows MemoRec to predict better classes within a package than structural features within a class. Moreover, by considering a longer list of recommended items, MemoRec obtains an increase in success rate.

5.2 RQ₂: How does the training data affect the performance of MemoRec?

We conducted similar experiments previously presented by measuring also the performance induced by the adoption of the dataset $\mathbf{D}_2$. As previously described in Sect. 4.2, $\mathbf{D}_1$ and

Table 7 Success rate for class recommendations, $k = \{1, 5, 10, 15, 20\}$, by considering the $\mathbf{D}_1$ dataset

k	SR@1	SR@10	SR@20
1	0.285	0.468	0.487
5	0.204	0.617	0.690
10	0.191	0.613	0.702
15	0.215	0.583	0.694
20	0.187	0.539	0.661

$\mathbf{D}_2$ are different in terms of size and quality. In particular, $\mathbf{D}_1$ contains different groups of similar metamodells. Each group is labeled to refer the application domain that the metamodells in the considered group are intended to describe. Thus, as done for answering RQ₁, we performed experiments by varying the number of neighbor nodes of the input metamodell and the value of N .

Tables 8(a) and 8(b) show the average success rates obtained by running the tenfold cross-validation technique to recommend structural features and classes, respectively, by using different cutoff values N .

According to Table 8, it is evident that using more neighbor metamodells to compute recommendations brings a better success rate for both datasets when the first recommended item is considered. An increasing number of neighbor k does not improve the success rate values when a longer list of recommendations is considered, i.e., SR@10 and SR@20. However, by using the randomly created dataset $\mathbf{D}_2$ success rate is lower than that of $\mathbf{D}_1$. For instance, with $\mathbf{D}_1$, MemoRec gets a success rate@1 of 0.159 and 0.202 when $k = 1$ and $k = 20$, respectively, whereas with $\mathbf{D}_2$ the corresponding values are 0.114 and 0.161. The same trend can also be seen with other cutoff values. As in the case of $\mathbf{D}_2$, MemoRec yields a better performance when we increase the cutoff value N . Take as an example, with $\mathbf{D}_2$ and $k = 20$, $N = 1$, the corresponding success rate is 0.178 which is less than a half of 0.373, the corresponding value when $N = 20$. In any case, the success rate related to the adoption of $\mathbf{D}_2$ is always lower than that of $\mathbf{D}_1$.

Table 8 Success rate, $k = \{1, 5, 10, 15, 20\}$, by comparing the adoption of $\mathbf{D}_1$ and $\mathbf{D}_2$

(a) For structural feature recommendations						
k	SR@1		SR@10		SR@20	
	$\mathbf{D}_1$	$\mathbf{D}_2$	$\mathbf{D}_1$	$\mathbf{D}_2$	$\mathbf{D}_1$	$\mathbf{D}_2$
1	0.153	0.114	0.394	0.328	0.502	0.397
5	0.181	0.150	0.406	0.363	0.489	0.452
10	0.202	0.161	0.419	0.340	0.501	0.413
15	0.200	0.170	0.392	0.327	0.494	0.395
20	0.202	0.178	0.362	0.320	0.479	0.373

(b) For class recommendations						
k	SR@1		SR@10		SR@20	
	$\mathbf{D}_1$	$\mathbf{D}_2$	$\mathbf{D}_1$	$\mathbf{D}_2$	$\mathbf{D}_1$	$\mathbf{D}_2$
1	0.285	0.147	0.468	0.307	0.487	0.331
5	0.204	0.173	0.617	0.493	0.691	0.589
10	0.191	0.150	0.613	0.445	0.702	0.571
15	0.215	0.147	0.583	0.397	0.694	0.521
20	0.187	0.141	0.539	0.362	0.661	0.478

Table 8(b) reports the success rate obtained with class recommendations by comparing the adoption of $\mathbf{D}_1$ and $\mathbf{D}_2$. The decrease in accuracy related to the adoption of $\mathbf{D}_2$ as shown in Table 8 is confirmed also in Table 8(b). To further study MemoRec's performance, we compute and report in Tables 9, 10, and 11 the precision, recall, and f-measure $\mathbf{D}_1$ and $\mathbf{D}_2$. For this setting, the number of recommended items N was varied from 1 to 10, attempting to examine the performance for a considerably long list of items. The value of k was varied of 1–20 with 5 as the step, considering a large number of neighbors.

Tables 9 and 10 confirm that by considering a curated dataset with more similar metamodels, MemoRec has better prediction performance than using a raw dataset. Moreover, MemoRec reaches better prediction performance in recommending structural features classes than classes packages.

Answer to RQ₂ The quality of the input data plays a key role in MemoRec's performance. Curated datasets with more similar metamodels allow MemoRec to improve its prediction performance, even if the size of such datasets is smaller than that of those randomly collected.

5.3 RQ₃: How do the encoding schemes affect the performance of MemoRec?

Before addressing this research question, we discuss the recommendations produced by MemoRec on the example shown in Fig. 8. More specifically, MemoRec has been applied on the Bibtex metamodel where the modeler is ask-

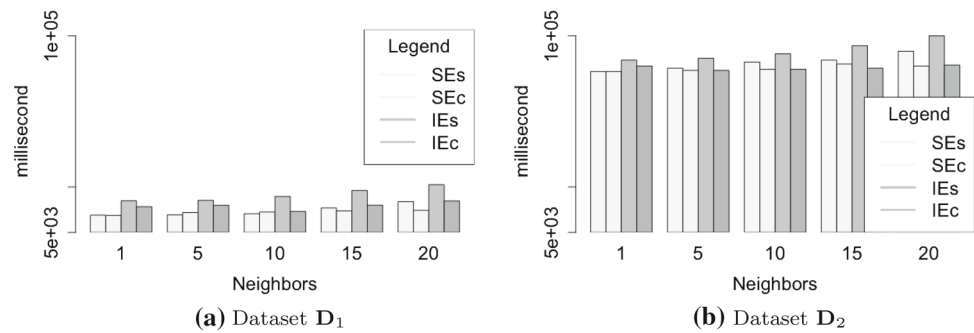
Table 9 Precision values

(a) For structural feature recommendations using $\mathbf{D}_1$					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
1	0.155	0.187	0.202	0.208	0.208
2	0.157	0.186	0.191	0.193	0.192
3	0.139	0.160	0.172	0.170	0.167
4	0.125	0.142	0.146	0.148	0.144
5	0.116	0.126	0.126	0.129	0.126
6	0.106	0.115	0.114	0.116	0.114
7	0.097	0.105	0.105	0.106	0.102
8	0.089	0.093	0.095	0.096	0.094
9	0.084	0.085	0.086	0.087	0.084
10	0.079	0.080	0.081	0.083	0.077

(b) For structural feature recommendations using $\mathbf{D}_2$					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
1	0.114	0.150	0.161	0.170	0.178
2	0.107	0.139	0.147	0.146	0.152
3	0.096	0.117	0.120	0.118	0.121
4	0.087	0.103	0.104	0.101	0.102
5	0.078	0.092	0.092	0.089	0.091
6	0.074	0.085	0.083	0.081	0.081
7	0.069	0.078	0.076	0.074	0.074
8	0.064	0.072	0.070	0.068	0.069
9	0.060	0.068	0.066	0.064	0.063
10	0.057	0.063	0.061	0.059	0.059

(c) For class recommendations using $\mathbf{D}_1$					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
1	0.236	0.193	0.204	0.198	0.180
2	0.235	0.213	0.179	0.182	0.164
3	0.218	0.198	0.162	0.159	0.142
4	0.209	0.181	0.156	0.149	0.138
5	0.196	0.175	0.151	0.144	0.124
6	0.185	0.168	0.150	0.134	0.114
7	0.177	0.160	0.147	0.131	0.112
8	0.166	0.148	0.142	0.130	0.114
9	0.163	0.151	0.144	0.127	0.110
10	0.156	0.158	0.146	0.125	0.105

(d) For class recommendations using $\mathbf{D}_2$					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
1	0.148	0.173	0.147	0.144	0.136
2	0.138	0.165	0.152	0.142	0.132
3	0.132	0.154	0.139	0.129	0.119
4	0.123	0.144	0.127	0.118	0.107
5	0.116	0.135	0.116	0.107	0.096
6	0.111	0.128	0.110	0.101	0.090
7	0.107	0.122	0.107	0.096	0.085
8	0.103	0.117	0.102	0.093	0.081

Fig. 7 Average execution time**Table 9** continued

(d) For class recommendations using D_2					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
9	0.099	0.113	0.099	0.090	0.079
10	0.096	0.112	0.096	0.087	0.076

Table 10 Recall values

(a) For structural feature recommendations using D_1					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
1	0.054	0.063	0.070	0.070	0.071
2	0.108	0.128	0.128	0.128	0.128
3	0.142	0.161	0.174	0.168	0.163
4	0.163	0.188	0.192	0.191	0.184
5	0.182	0.203	0.202	0.204	0.195
6	0.195	0.216	0.214	0.217	0.209
7	0.205	0.227	0.225	0.225	0.214
8	0.214	0.228	0.235	0.233	0.224
9	0.228	0.233	0.239	0.239	0.226
10	0.237	0.242	0.248	0.250	0.228

(b) For structural feature recommendations using D_2					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
1	0.037	0.051	0.057	0.060	0.063
2	0.070	0.095	0.103	0.102	0.107
3	0.092	0.115	0.123	0.120	0.124
4	0.108	0.133	0.139	0.134	0.136
5	0.121	0.145	0.150	0.145	0.148
6	0.135	0.159	0.160	0.154	0.155
7	0.146	0.169	0.168	0.165	0.164
8	0.153	0.177	0.177	0.171	0.172
9	0.162	0.188	0.185	0.178	0.177
10	0.169	0.194	0.190	0.183	0.182

(c) For class recommendations using D_1					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
1	0.032	0.039	0.048	0.049	0.048
2	0.056	0.074	0.084	0.088	0.088
3	0.067	0.091	0.102	0.111	0.109

Table 10 continued

(c) For class recommendations using D_1					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
4	0.077	0.099	0.112	0.119	0.118
5	0.083	0.109	0.120	0.132	0.127
6	0.090	0.118	0.132	0.138	0.134
7	0.099	0.124	0.143	0.149	0.144
8	0.103	0.127	0.152	0.157	0.152
9	0.109	0.137	0.162	0.163	0.158
10	0.114	0.149	0.172	0.169	0.162

(d) For class recommendations using D_2					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
1	0.018	0.030	0.030	0.032	0.032
2	0.034	0.058	0.062	0.063	0.061
3	0.044	0.075	0.078	0.078	0.077
4	0.050	0.085	0.087	0.085	0.083
5	0.054	0.092	0.092	0.090	0.087
6	0.058	0.099	0.098	0.096	0.092
7	0.062	0.105	0.106	0.101	0.095
8	0.065	0.110	0.111	0.106	0.099
9	0.069	0.116	0.116	0.112	0.105
10	0.072	0.123	0.121	0.116	0.109

ing for the recommendation for two different contexts, i.e., the *Article* class (rounded in red) and the *Bibtex* package (rounded in blue). Because of the different context's nature, we show the recommendations to *Article* class by considering the IE_s and SE_s encodings. In contrast, the IE_c and SE_c encodings are discussed by considering the *Bibtex* package as the active context. We briefly summarize the four proposed schemes in Sect. 3.1 as follows. SE_s consists of *class-structural feature* pairs for each structural feature directly defined within a class while IE_s also includes structural feature inherited from the superclasses; IE_c consists of *package-class* pairs while SE_c flattens packages within a default artificial package.

Table 12 shows the list of top-10 recommended items for each encoding scheme, i.e., SE_c , IE_c , SE_s , and SE_c . As

Table 11 F-measure values

(a) For structural feature recommendations using $\mathbf{D}_1$					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
1	0.080	0.094	0.104	0.105	0.106
2	0.128	0.152	0.153	0.154	0.154
3	0.140	0.160	0.173	0.169	0.165
4	0.141	0.162	0.166	0.167	0.162
5	0.142	0.155	0.155	0.158	0.153
6	0.137	0.150	0.149	0.151	0.148
7	0.132	0.144	0.143	0.144	0.138
8	0.126	0.132	0.135	0.136	0.132
9	0.123	0.125	0.126	0.128	0.122
10	0.119	0.120	0.122	0.125	0.115
(b) For structural feature recommendations using $\mathbf{D}_2$					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
1	0.056	0.076	0.084	0.089	0.093
2	0.085	0.113	0.121	0.120	0.126
3	0.094	0.116	0.121	0.119	0.122
4	0.096	0.116	0.119	0.115	0.117
5	0.095	0.113	0.114	0.110	0.113
6	0.096	0.111	0.109	0.106	0.106
7	0.094	0.107	0.105	0.102	0.102
8	0.090	0.102	0.100	0.097	0.098
9	0.088	0.100	0.097	0.094	0.093
10	0.085	0.095	0.092	0.089	0.089
(c) For class recommendations using $\mathbf{D}_1$					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
1	0.056	0.065	0.078	0.079	0.076
2	0.090	0.110	0.114	0.119	0.115
3	0.102	0.125	0.125	0.131	0.123
4	0.113	0.128	0.130	0.132	0.127
5	0.117	0.134	0.134	0.138	0.125
6	0.121	0.139	0.140	0.136	0.123
7	0.127	0.140	0.145	0.139	0.126
8	0.127	0.137	0.147	0.142	0.130
9	0.131	0.144	0.152	0.143	0.130
10	0.132	0.153	0.158	0.144	0.127
(d) For class recommendations using $\mathbf{D}_2$					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
1	0.032	0.051	0.050	0.052	0.052
2	0.055	0.086	0.088	0.087	0.083
3	0.066	0.101	0.100	0.097	0.094
4	0.071	0.107	0.103	0.099	0.093
5	0.074	0.109	0.103	0.098	0.091
6	0.076	0.112	0.104	0.098	0.091
7	0.079	0.113	0.106	0.098	0.090
8	0.080	0.113	0.106	0.099	0.089

Table 11 continued

(d) For class recommendations using $\mathbf{D}_2$					
N	$K = 1$	$K = 5$	$K = 10$	$K = 15$	$K = 20$
9	0.081	0.114	0.107	0.100	0.090
10	0.082	0.117	0.107	0.099	0.090

described in Sect. 3.1, the type of the active context discriminates which encoding MemoRec needs to adopt. By observing the outcomes of Table 12, it can be seen that using IE_s and SE_s produces diverse recommended items. The result provided by IE_s seems to contain more generic recommendations, e.g., id, actor, attachment, to name a few, while SE_s provides more focused recommended structural features. This intuition is confirmed by manually analyzing significant recommendation examples. Though both IE_s and SE_s provide useful suggestions to the modeler, the augmented information of SE_c allows MemoRec to predict more specific structural features. Referring to the scores obtained with IE_c and SE_c , we conclude that the results depend on the similarities among the training metamodels. By flattening package structure, SE_c mainly uses class names to compute similarities, while IE_c includes also package structure. So, when a similar metamodel is identified, SE_c can recommend classes that belong to any package.

To answer RQ₃, we conducted experiments on $\mathbf{D}_1$ by comparing pairwise the encoding schemes, i.e., SE_s versus IE_s and SE_c versus IE_c . As we mentioned before, we should select a suitable number of neighbor metamodels to compute recommendations to maintain a trade-off between efficiency and effectiveness. Thus, to simplify the evaluation, we set the number of neighbors k to 5.² The evaluation metrics computed by the encoding schemes for structural features and classes are reported in Tables 13a and 13b, respectively.

Table 13a demonstrates an evident outcome: by using IE_s as the encoding scheme, we obtain a better prediction performance than that when using SE_s . For instance, given $N = 1$, we get a success rate of 0.241 and 0.181 for IE_s and SE_s encodings, respectively. Similarly by other evaluation metrics, i.e., precision and recall, IE_s helps achieve a superior performance. When we consider larger cutoff values, i.e., $N = \{5, 10, 15, 20\}$, IE_s is always beneficial to the recommendation outcomes, as it brings in higher quality indicators. Take as an example, with $N = 20$, using IE_s yields a success rate of 0.604, which is much higher than 0.489, the corresponding value for SE_s .

Next, we analyze the recommendations by using SE_c and IE_c as shown in Table 13b. For success rate and precision, using IE_c helps MemoRec perform better than using SE_c .

² For each dataset and N value, the average success rate is higher with $k = 5$ than that of other values of k .

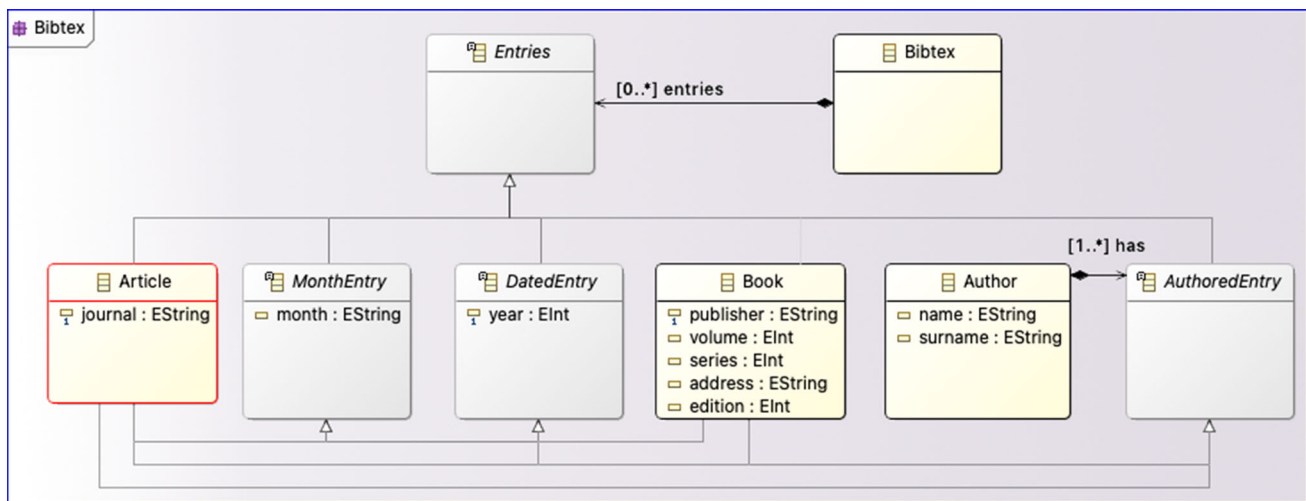


Fig. 8 The Bibtex metamodel

Table 12 Predicted recommendations for the *Article* metaclass and Bibtex package of the metamodel in Fig. 8

	IE _s	SE _s
Article metaclass	ContainerPage	Address
	id	edition
	toolspecifics	journal
	abstractPrefix	month
	abstractText	note
	acceptDirty	number
	actor	pages
	address	publisher
	attachment	series
	authorName	volume
	IE _c	SE _c
Bibtex package	AbstractCondition	Abstract
	Assign	CommitteeMember
	AssignExtra	AcademiaOrganization
	AttributeCondition	AcademicEvent
	Book	Academic_Institution
	ConditionalState	AcceptRating
	ConditionalTransition	Acceptance
	Editor	AcceptedPaper
	EmphasisValue	AccommodationPlace
	Event	Account

However, IE_c negatively impacts on the recall values. In our opinion, it is due to the flattening operation which affects the similarity function, making metamodels too similar. In this case, the recommender engine is limited to suggest the most common classes. For instance, because of metamodeling best-practice, *NamedElement* is commonly used in a metamodel. This impacts on success rate and precision, but

recall goes down because the recommended items do not depend on the metamodel context.

Through the experiment, we see that a suitable encoding scheme fosters better prediction performance. In this respect, we believe that the introduction of natural language processing (NLP) steps, i.e., stemming, lemmatization, and stop words removal, can boost up the accuracy of MemoRec. In particular, preprocessing steps can be employed to reduce the usage of different terms with very close semantics and, thus, to increase the corresponding term usages. For instance, the word “*reference*” and its plural form “*references*” are two conjugations of the same noun. Even though the two words have the same semantics, currently, MemoRec does not match those two terms and considers them different by negatively affecting the resulting MemoRec performance. We consider the integration of NLP techniques as our future work.

The average execution time among tenfolds on various values of k for both datasets is depicted in Fig. 7a and Fig. 7b. It can be seen that while IE_s helps MemoRec achieve a good prediction performance, it sustains a high computational complexity, resulting in prolonged execution time. This is understandable since compared to other techniques, the encoding scheme incorporates more information from metamodels for its computation.

Answer to RQ₃ Inherited structural features (IE_s) enable MemoRec to achieve a superior performance compared to using SE_s, despite a higher computational complexity. With IE_s, MemoRec predicts better structural features within a class than classes within a package.

Table 13 Success rate, precision and recall for class recommendations

(a) Using SE_s and IE_s encodings						
N	SR@N		Precision@N		Recall@N	
	IE_s	SE_s	IE_s	SE_s	IE_s	SE_s
1	24.074	18.113	0.241	0.181	0.078	0.061
5	39.074	33.207	0.119	0.121	0.169	0.196
10	48.333	40.566	0.084	0.078	0.241	0.245
15	55.926	46.603	0.068	0.059	0.291	0.271
20	60.370	48.867	0.058	0.047	0.331	0.284

(b) Using SE_c and IE_c encodings						
N	SR@N		Precision@N		Recall@N	
	IE_c	SE_c	IE_c	SE_c	IE_c	SE_c
1	22.593	20.370	0.226	0.204	0.014	0.037
5	56.296	53.333	0.220	0.196	0.061	0.117
10	65.926	61.667	0.194	0.163	0.100	0.152
15	70.370	66.667	0.186	0.158	0.139	0.189
20	73.333	69.074	0.167	0.140	0.159	0.210

5.4 Threats to validity

In this section we give a discussion of threats, which might harm the validity of the performed experiments. In particular, we discuss threats with respect to internal and external validity as follows.

Internal validity Such threats refer to internal factors that might affect the outcomes of the performed experiments. A possible threat is represented by the datasets that have been used for the experiments. We mitigated such a threat by using two completely different datasets, and one of them has been randomly created without performing any data curation activities. Another internal threat to validity factor is represented by the adopted encoding schemes. Also in this case, we mitigated the issue by employing different encoding schemes. However, by considering data and encoding dimensions, we managed to identify distinctive characteristics of the approach that resulted to be valid independently from the adopted encoding schemes and input data sets, i.e., graph builder, similarity calculator, and recommendation engine.

External validity It is related to factors that can affect the generalizability of our findings, by possibly making the obtained results not valid outside the scope of this study. We mitigated the issue by evaluating MemoRec in different scenarios, with the aim of simulating several usages of the systems, e.g., by varying the number of neighbor metamodels, and the size of the list of recommended items. Another threat to validity can be the fact that currently, we do not consider the sequences of actions that are operated to lead to a given metamodel. We believe that alternative approaches

like long short-term memory (LSTM) [16] can be a possible candidate to produce recommendations that rely on creation sequences. Moreover, it is crucial to investigate how modelers perceive MemoRec. In this respect, we plan to conduct a user study where human evaluators are asked to give their assessment for recommendations provided by MemoRec.

6 Discussion

Mussbacher et al. [25] proposed a conceptual framework to characterize and compare approaches for intelligent modeling assistance (IMA). The proposed assessment grid consists of nine properties, namely *Model*, *Autonomy*, *Relevance*, *Confidence*, *Trust*, *Explainability*, *Quality Degree*, *Timeliness*, and *Quality regarding external sources*. We provide a qualitative discussion of MemoRec with respect to such properties as follows.

With *Model*, the authors [25] refer to the quality level of the models recommended by the considered IMA. Such a quality is measured with syntactic, semantic, and pragmatic quality. Concerning such a dimension, MemoRec ensures syntactic quality as long as it is trained with syntactically correct artifacts.

With the *Autonomy* property, authors refer to what extent the IMA under analysis is autonomous in gathering context information or user feedback without any user intervention. Such a property is mainly related to the tool gearing the considered IMA. MemoRec cannot be analyzed concerning the *Autonomy* dimension because, at this stage, we focused on the algorithmic part of the approach and deferred its integration in a supporting environment as a next step.

The *Relevance* property is related to the degrees of precision and recall of the recommendations provided by the adopted IMA with respect to modeler intentions. As shown in Sect. 5, MemoRec has been evaluated by resembling different configurations, and the obtained accuracy is satisfactory, and it very much depends on the quality of the training data.

With the *Confidence* property, Mussbacher et al. [25] aim at measuring how often the IMA under analysis provides a confidence value for the recommended items. Similarly to all the analyzed IMAs [25], MemoRec does not provide any confidence value for the recommended metamodel elements. Indeed, this represents an interesting and important improvement for MemoRec.

Trust has been defined as “the perception that the modeler has about the quality of an IMA.” This represents an important property that we plan to assess once we have to evaluate the quality of the supporting tool integration of MemoRec in an existing IDE, e.g., Eclipse.

The *Explainability* property is another essential characteristic that is related to the emerging *Explainable AI* research field [13], which aims at existing AI techniques and tools to

make produced outcomes understood by humans. The current version of MemoRec cannot provide any explanations related to recommended elements. An interesting extension can be complementing recommended metamodel elements with the sources of the most representative metamodels that triggered the given recommendations.

The *Quality Degree* dimension measures “the degree of excellence of the IMA to address the needs of a modeler.” Similar to all the approaches that have been analyzed [25], MemoRec relies on external sources to provide modelers with recommendations that are valid for the active context.

Since *Timeliness* refers to the user satisfaction for a given IMA, such a quality metric cannot be assessed for MemoRec at this stage. Instead, we plan to evaluate it once MemoRec is integrated into an existing IDE, like Eclipse.

The *quality regarding external sources* property refers to the quality of the IMA under analysis concerning its external sources. As shown in Fig. 2, MemoRec is repository independent as long as it is possible to download the available modeling artifacts. However, as also discussed in Sect. 5.2, being MemoRec a data-driven approach, the quality of the mined metamodels has an impact on the performance of MemoRec. Consequently, if the system is trained with low-quality metamodels, also recommendations would be of limited quality.

7 Related work

7.1 Existing modeling assistants

The Extremo tool [24] has been proposed to assist modelers in a platform-independent way. By relying on miscellaneous resources excerpted from the context, it creates shared data employed to develop a flexible query mechanism. This querying system is used to explore and find useful entities that help the modeler complete the model under construction. To assess the quality of the work, Extremo has been used to implement a DSL for the financial domain and validate its soundness through different use cases. The tool has been fully integrated into the Eclipse IDE as a plugin. In contrast with MemoRec that extracts the query directly from the metamodel under development, Extremo requires a modeler to perform custom or pre-defined queries to search for information chunks.

Papyrus [11] is a system to support domain-specific model specification by exploiting UML profiles. The mapping between each profile’s metaclass and real Java classes is performed using EMF generator model utilities. Furthermore, the tool embeds a palette and a context menu to graphically specify the selected metaclasses. Though the results are promising, more features could be added to enrich the experience, e.g., proactive triggering of recommenda-

tions or fine-grained customizations using EMF utilities. By transforming UML profiles to EMF metamodels, Papyrus supports the development of domain-specific environments. Although the generated domain-specific environments facilitate editing a model, Papyrus does not support the modeler with suggestions to complete the input profile from which the EMF metamodel is generated nor edit a model that conforms to those specifications.

Batot and Sahraoui [2] introduced a modeling assistant based on a multi-objective optimization problem (MOOP). A well-founded evolutionary algorithm, namely NSGA-II, is employed to obtain representative models using an initial set provided by the user. According to the Pareto optimality definition, the algorithm is able to solve the MOOP to find relevant candidates. In this way, NSGA-II retrieves partial models to be completed by expert-domain users who can personalize the obtained results by selecting different coverage degree or changing pre-defined minimality criteria. To assess the quality of the work, the proposed NSGA-II adaptation was compared with random and mono-objective functions. The experimental results showed that the MOOP adaptation outperforms the baselines. Different from MemoRec that suggests possible metamodel elements, the approach in [2] generates a set of models for various MDE tasks, e.g., testing automated learning.

López-Fernández et al. [23] presented an example-driven tool to recommend a complete metamodel starting from model fragments specified by graphical tools, e.g., Visio, PowerPoint, Dia. The tool extracts untyped model fragments using initial examples as the starting point. Then, it infers an agnostic metamodel from them that the modeler can then enrich. In contrast with MemoRec, metamodels are generated by an iterative and inductive process where model fragments are given either sketched by domain experts using drawing tools or by a compact textual notation.

AVIDA-MDE [15] extends the original AVIDA tool and facilitates the generation of behavioral models starting from the requirements’ specification. At the beginning of the process, an instinctual knowledge is formed by considering state diagrams, their inner elements, and the alphabet to specify such models. By relying on such information, AVIDA-MDE constructs new transitions and propose different behavior models to support scenarios and meet the constraints specified in the initial phase by the user. The approach was evaluated using a robot navigation system as the testing scenario. AVIDA-MDE aims at generating a set of behavioral models starting from the given parameters, whereas MemoRec recommends additional model elements starting from the context consisting of the metamodel under development.

A model assistant based on clone detection has been recently envisioned [41]. The envisioned prototype is able to edit incomplete input model as well as propose fine-grain operations on the model itself. The approach makes

use of Simulink Virtual Modeling Assistant (SimVMA), a well-founded technique to detect model clones. SimVMA is capable of finding all possible intersections between the initial metamodel and the clones belonging to the knowledge base. By exploiting the Type-3 clone similarity, the system can perform the two recommended types, i.e., retrieving complete models or single operation suggestions. By using similar metamodels, MemoRec applies a context-aware collaborative filtering technique using a given number of similar metamodels to predict additional model elements.

MAR [22] employs a query-by-example approach to search for similar metamodels/models. First, model structure is encoded as bags of paths before being indexed and stored on Apache HBase. Given a model, MAR uses it as a query to search for similar artifacts using a similarity score. Though modelers can learn by inspecting similar metamodels, they have to manually examine the results to extract useful information.

Sen et al. [38] proposed a model assistant based on constraint logic program (CLP) to support the definition of domain-specific models in the modeling framework Atom³. Given a partial model, the proposed tool is able to synthesize the complete model by relying on several constraints specified in Prolog. The designer can eventually ask for additional recommendations using the generated domain-specific model editor. Similarly, an extension of diagram predicate framework (DPF) has been proposed to rewrite partial models by adding new elements graphically [32]. In particular, the proposed framework grants the compliance of the edited model using several termination rules adapted from the layered graph grammar technique. Differently from MemoRec that mines existing metamodels to predict additional model elements, the proposed approach does not use information from existing models, but makes use of (i) the notation of the modeler under development, (ii) the constraints expressed on this metamodel, and (iii) the partial model built by a domain expert to generate a visual model editor for the DSML supporting recommendations for possible completions.

The ASketch tool [43] supports the completion of Alloy partial model with holes using automated analysis. First, the input interpreter parses the partial model to generate possible candidates. Afterward, these model fragments are encoded with the partial model and AUnit test files. Thus, ASketch can find possible solutions in a large search space by relying on an SAT solver. ASketch fills an alloy partial model with concrete candidate fragments such that predefined tests, i.e., unit testing, test execution. Therefore, the final model fragments are not extracted from existing model/metamodel, which is actually done by MemoRec.

Recently, we presented MORGAN [9], a recommender system based on a graph neural network (GNN) to assist modelers in performing the specification of metamodels and models. Similar to MemoRec, MORGAN makes use of

tailored model and metamodel parsers to excerpt relevant information in textual data format. Then, the encoder builds the graphs from the text produced by the parser. Finally, the generated graphs feed a GNN-based engine to compute additional metamodel or model parts. Unlike MemoRec, MORGAN does not need an active context where the recommender suggests additional elements. At the same time, MemoRec attempts to get very related recommendations to the context where the modeler is working on the definition. For this reason, we cannot directly compare MORGAN with MemoRec.

Our work distinguishes itself from the studies mentioned above as it can provide missing classes and structural features for a metamodel under development, exploiting a context-aware collaborative filtering technique. In addition, we anticipate that applying natural language processing techniques can help MemoRec improve the prediction performance, and we consider the issue in our future work.

7.2 Code recommender systems

In the context of open-source software, developing new systems by reusing existing components raises relevant challenges in (i) searching for relevant modules; and (ii) adapting the selected components to meet pre-defined requirements. To this end, recommender systems in software engineering have been developed to support developers in their daily tasks [7,33]. Such systems have gained traction in recent years as they can provide developers with a wide range of valuable items, including code snippets [28,30], tags/topics [10], third-party libraries [27], documentation [34], to mention but a few.

Sourcerer [20] performs code search in large-scale repositories by exploiting different components. The first component is the crawler, which automatically downloads repositories to build a knowledge base. Then, it parses the source code to represent it as a database entity. Additionally, Apache Lucene and fingerprint are used to support keyword-base search and structural representation of the repository, respectively. Finally, the ranker retrieves the most relevant results.

StackOverflow has been exploited to enrich code queries, with the aim of getting relevant source code. In particular, FaCoY [17] is a code-to-code search engine that recommends relevant GitHub snippets to a project being developed. It is based on an alternate query technique to augment the possible retrieved results. The initial query is built from StackOverflow posts, and the additional query is performed directly on GitHub local repositories to deliver final recommendation items.

Differently from the work that proposes tools being able to provide developers with specific recommendations, Koro-taev et al. [19] introduce a GRU-based recurrent neural network (RNN) to build a universal recommender system. To

this end, the approach supports the recommendation phase using a client-server architecture equipped with different components. The data collection and processing phases are conducted, taking into consideration user's behavior. The data mining module is used to feed a GRU-based RNN. To support user profiling, the approach uses ontologies by building an external knowledge representation module. The proposed network outperforms the long short-term memory (LSTM) technique concerning accuracy.

MemoRec has been built following a series of recommender systems developed through the CROSSMINER project [7]. Rather than recommending source code, or libraries, MemoRec provides modelers with artifacts related to metamodeling activities, using a collaborative filtering technique. Such a technique has been successfully exploited to build recommender systems to suggest API calls [30] and third-party libraries [27]. MemoRec processes input data with four different encoding techniques. Its internal design is also tailored to compute similarity among metamodels in an efficient way.

7.3 Application of ML in MDE

In recent years, we have witnessed a proliferation of artificial intelligence (AI) in various aspects of human life. In the MDE domain, though learning algorithms have been successfully applied to tackle various issues, the adoption of AI and machine learning (ML) techniques in this domain is still in its infancy. This section recalls some of the most important work in this topic.

Mussbacher et al. [25] conducted an initial investigation on intelligent modeling assistants (IMAs) using a comprehensive assessment grid. To elicit critical IMAs features, the authors analyzed the well-founded reference framework for intelligent modeling assistance (RF-IMA). The main finding of the work is that existing IMAs obtain low scores in the extracted features and they can be further enhanced in terms of performance as well as in the underpinning structure.

Breuker [3] reviews the main modeling languages used in ML as well as inference algorithms and corresponding software implementations. The aim of this work is to explore the opportunities of defining a DSML for probabilistic modeling. To allow developers to design solutions that solve machine learning-based problems by automatically generating code for other technologies as a transparent bridge, a language and a technology-independent development environment are introduced in [12]. A similar tool named OptiML has been proposed [42], aiming to bridge the gap between ML algorithms and heterogeneous hardware to provide a productive programming environment.

In collaborative modeling, Barriga et al. [1] propose the adoption of an unsupervised and reinforcement learning (RL) approach to repair broken models, which have been cor-

rupted because of conflicting changes. The main intent is to potentially reach model repairing with human-quality without requiring supervision.

AURORA [29] can be considered as the first attempt to classify metamodels exploiting a machine learning algorithm. The approach is built on top of a neural network to learn from labeled metamodels and classify unlabeled data. Despite its simplicity, the tool is efficient and on a small dataset, it classifies the metamodels, obtaining high prediction accuracy. In a recent work [31], we further improved the performance by employing a convolutional neural network to classify metamodels.

Heterogeneity issues in customizable recommender systems have been analyzed by involving two different use cases [39]. The participants had to tune the system according to their preferences. In the first session, users configured a travel recommender by means of different facets of the trip, i.e., costs, food, and location. The second use case involved a personal exercise recommender system for the training activity. The results show that even homogeneous groups of users select different system configurations. Thus, a tailored recommender system might consider the mental model of the target users, namely their preferences and custom algorithms.

Blended recommending [21] introduces a similar strategy embedded in a movie recommender. It implements several filtering techniques used in the domain, i.e., content-based filtering and collaborative filtering. Using the blended recommending strategy, users can specify a recommendation algorithm as well as refine its parameters in a hybrid filtering fashion. In this landscape, our work aims to cope with these challenges by promoting the adoption of a low-code platform. To our best knowledge, this is the first attempt to use such a technology in this domain.

A recent work [41] envisioned a new idea for supporting the modeler with step-wise guidance or entire model examples. The proposed approach involves model clone detection techniques to find similar metamodels to the one that the modeler is defining. In contrast to our approach, the discovery of similar metamodels does not include language syntaxes, but it relies on the Simone model clone detector.

8 Conclusions and future work

In this paper, we introduced MemoRec, a novel approach that uses a context-aware collaborative filtering technique to support the modeler in completing the specification of a metamodel. By encoding metamodels and their contents in four different schemes, we built rating matrices and applied a syntactic-based similarity function to predict missing items, i.e., classes and structural features. An evaluation on two independent datasets, i.e., $\mathbf{D}_1$ and $\mathbf{D}_2$, and four encoding schemes, i.e., SE_s , IE_s , SE_c , and IE_c , exploiting tenfold cross-

validation demonstrates that the tool is able to provide decent recommendations.

We plan to extend MemoRec by adding other similarity functions, e.g., structural and semantic based methods. Moreover, we can improve the encoding schemes by introducing natural language preprocessing (NLP) techniques. We will augment additional information to the recommendation outcomes, e.g., type, cardinality. Afterward, we are going to conduct a proper user study with the involvement of modelers to evaluate the usability of MemoRec. Last but not least, now that we have validated the algorithmic accuracy of the proposed technique, we will integrate the conceived tool into the Eclipse IDE, providing modelers with supports embedded in their development environment.

Acknowledgements The research described in this paper has been partially supported by the AIDOaRT Project, which has received funding from the European Union's H2020-ECSEL-2020, Federal Ministry of Education, Science and Research, Grant Agreement n° 101007350. We thank the anonymous reviewers for their valuable comments and suggestions that helped us improve the paper.

Funding Open access funding provided by Università degli Studi dell'Aquila within the CRUI-CARE Agreement

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Barriga, A., Rutle, A., Heldal, R.: Automatic model repair using reinforcement learning. In: Hebig R., Berger T. (eds.) Proceedings of MODELS 2018 Workshops co-located with ACM/IEEE 21st International Conference on Model Driven Engineering Languages and Systems (MODELS 2018), Copenhagen, Denmark, Volume 2245 of CEUR Workshop Proceedings, pp. 781–786. CEUR-WS.org (2018)
2. Batot, E., Sahraoui, H.: A generic framework for model-set selection for the unification of testing and learning MDE tasks. In: Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems-MODELS '16, Saint-malo, France, pp. 374–384. ACM Press (2016)
3. Breuker, D.: Towards model-driven engineering for big data analytics: an exploratory analysis of domain-specific languages for machine learning. In: 47th Hawaii International Conference on System Sciences, pp. 758–767 (2014)
4. Chen, A.: Context-aware collaborative filtering system: predicting the user's preference in the ubiquitous computing environment. In: First International Conference on Location- and Context-Awareness, pp. 244–253. Springer, Berlin (2005)
5. Di Rocco, J., Di Ruscio, D., Di Sipio, C., Nguyen, P., Rubei, R.: Topfilter: an approach to recommend relevant github topics. In: Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), ESEM '20, New York, NY, USA. Association for Computing Machinery (2020)
6. Di Rocco, J., Di Ruscio, D., Di Sipio, C., Nguyen, P., Rubei, R.: Topfilter: an approach to recommend relevant github topics. In: Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), pp. 1–11 (2020)
7. Di Rocco, J., Di Ruscio, D., Di Sipio, C., Nguyen, P.T., Rubei, R.: Development of recommendation systems for software engineering: the crossminer experience. *Empir. Softw. Eng.* **26**(4), 1–40 (2021)
8. Di Rocco, J., Di Ruscio, D., Iovino, L., Pierantonio, A.: Mining metrics for understanding metamodel characteristics. In: Proceedings of the 6th International Workshop on Modeling in Software Engineering, MiSE 2014, New York, NY, USA, pp. 55–60. Association for Computing Machinery (2014)
9. Di Rocco, J., Di Sipio, C., Di Ruscio, D., Nguyen, P.T.: A GNN-based recommender system to assist the specification of metamodels and models. In: ACM/IEEE 24th International Conference on Model Driven Engineering Languages and Systems (MODELS), pp. 1–11 (2021)
10. Di Sipio, C., Rubei, R., Di Ruscio, D., Nguyen, P.T.: A multinomial naïve Bayesian (mnb) network to automatically recommend topics for github repositories. In: Proceedings of the Evaluation and Assessment in Software Engineering, EASE '20, New York, NY, USA, pp. 71–80. Association for Computing Machinery (2020)
11. Dupont, G., Mustafiz, S., Khendek, F., Toeroe, M.: Building Domain-Specific Modelling Environments with Papyrus: An Experience Report, p. 8 (2018)
12. García-Díaz, V., Espada, J.P., Bustelo, B.C.P.G., Lovelle, J.M.C.: Towards a standard-based domain-specific platform to solve machine learning-based problems. *IJIMAI* **3**(5), 6–12 (2015)
13. Giannotti, F.: Explainable machine learning for trustworthy AI. In: Rocha A.P., Steels L., van den Herik H.J. (eds.) Proceedings of the 13th International Conference on Agents and Artificial Intelligence, ICAART 2021, vol. 1, Online Streaming, p. 11. SCITEPRESS (2021)
14. GitHub REST API v3. <https://developer.github.com/v3/>. Last accessed 16 June 2020
15. Goldsby, H.J., Cheng, B.H.: Avida-MDE: a digital evolution approach to generating models of adaptive software behavior. In: Proceedings of the 10th Annual Conference on Genetic and Evolutionary Computation-GECCO '08, Atlanta, GA, USA, pp. 1751. ACM Press (2008)
16. Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Comput.* **9**(8), 1735–1780 (1997)
17. Kim, K., Kim, D., Bissyandé, T.F., Choi, E., Li, L., Klein, J., Traon, Y.L.: Facoy: a code-to-code search engine. In: Proceedings of the 40th International Conference on Software Engineering-ICSE '18, Gothenburg, Sweden, pp. 946–957. ACM Press (2018)
18. Kohavi, R., et al.: A study of cross-validation and bootstrap for accuracy estimation and model selection. In: *Ijcai*, Montreal, Canada, vol. 14, pp. 1137–1145 (1995)
19. Korotaev, A., Lyadova, L.: Method for the development of recommendation systems, customizable to domains, with deep GRU network. In: Proceedings of the 10th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management, Seville, Spain, pp. 231–236. SCITEPRESS - Science and Technology Publications (2018)

20. Linstead, E., Bajracharya, S., Ngo, T., Rigor, P., Lopes, C., Baldi, P.: Sourcerer: mining and searching internet-scale software repositories. *Data Min. Knowl. Discov.* **18**(2), 300–336 (2009)
21. Loepp, B., Herrmann, K., Ziegler, J.: Blended recommending: integrating interactive information filtering and algorithmic recommender techniques. In: *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems—CHI '15*, Seoul, Republic of Korea, pp. 975–984. ACM Press (2015)
22. López, J.A.H., Cuadrado, J.S.: Mar: a structure-based search engine for models. In: *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, MODELS '20*, New York, NY, USA, pp. 57–67. Association for Computing Machinery (2020)
23. López-Fernández, J.J., Cuadrado, J.S., Guerra, E., de Lara, J.: Example-driven meta-model development. *Softw. Syst. Model.* **14**(4), 1323–1347 (2015)
24. Mora Segura, A., de Lara, J.: Extremo an eclipse plugin for modelling and meta-modelling assistance. *Sci. Comput. Program.* **180**, 71–80 (2019)
25. Mussbacher, G., Combemale, B., Abrahão, S., Bencomo, N., Burguño, L., Engels, G., Kienzle, J., Kühn, T., Mosser, S., Sahraoui, H., Weyssow, M.: Towards an assessment grid for intelligent modeling assistance. In: *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, MODELS '20*, New York, NY, USA. Association for Computing Machinery (2020)
26. Babur, Ö.: A labeled Ecore metamodel dataset for domain clustering. <https://doi.org/10.5281/zenodo.2585456> (2019)
27. Nguyen, P.T., Di Rocco, J., Di Ruscio, D., Di Penta, M.: Cross-Rec: supporting software developers by recommending third-party libraries. *J. Syst. Softw.* **161**, 110460 (2019)
28. Nguyen, P.T., Di Rocco, J., Di Ruscio, D., Ochoa, L., Degueule, T., Di Penta, M.: FOCUS: a recommender system for mining api function calls and usage patterns. In: *Proceedings of the 41st International Conference on Software Engineering, ICSE '19*, Piscataway, NJ, USA, pp. 1050–1060. IEEE Press (2019)
29. Nguyen, P.T., Di Rocco, J., Di Ruscio, D., Pierantonio, A., Iovino, L.: Automated classification of metamodel repositories: a machine learning approach. In: *ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems (MODELS)*, pp. 272–282 (2019)
30. Nguyen, P.T., Di Rocco, J., Di Sipio, C., Di Ruscio, D., Di Penta, M.: Recommending api function calls and code snippets to support software development. *IEEE Trans. Softw. Eng.* 1–1 (2021)
31. Nguyen, P.T., Di Ruscio, D., Pierantonio, A., Di Rocco, J., Iovino, L.: Convolutional neural networks for enhanced classification mechanisms of metamodels. *J. Syst. Softw.* **172**, 110860 (2020)
32. Rabbi, F., Lamo, Y., Yu, I.C., Kristensen, L.M.: A diagrammatic approach to model completion. In: *AMT@MoDELS* (2015)
33. Robillard, M.P., Maalej, W., Walker, R.J., Zimmermann, T. (eds.): *Recommendation Systems in Software Engineering*. Springer, Berlin (2014)
34. Rubei, R., Di Sipio, C., Nguyen, P.T., Di Rocco, J., Di Ruscio, D.: Postfinder: mining stack overflow posts to support software developers. *Inf. Softw. Technol.* **127**, 106367 (2020)
35. Sarwar, B., Karypis, G., Konstan, J., Riedl, J.: Item-based collaborative filtering recommendation algorithms. In: *10th International Conference on World Wide Web*, pp. 285–295. ACM, New York (2001)
36. Schafer, J.B., Frankowski, D., Herlocker, J., Sen, S.: Collaborative Filtering Recommender Systems, pp. 291–324. Springer, Berlin (2007)
37. Schmidt, D.C.: Guest editor's introduction: model-driven engineering. *Computer* **39**(2), 25–31 (2006)
38. Sen, S., Baudry, B., Vangheluwe, H.: Towards domain-specific model editors with automatic model completion. *Simulation* **86**(2), 109–126 (2010)
39. Solomon, J.: Heterogeneity in customization of recommender systems by users with homogenous preferences. In: *Proceedings of the CHI Conference on Human Factors in Computing Systems*, San Jose, California, USA, pp. 4166–4170. ACM (2016)
40. Steinberg, D., Budinsky, F., Merks, E., Paternostro, M.: *EMF: Eclipse Modeling Framework*. Pearson Education, London (2008)
41. Stephan, M.: Towards a cognizant virtual software modeling assistant using model clones. In: *IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, pp. 21–24. IEEE (2019)
42. Sujeeth, A., Lee, H., Brown, K., Rompf, T., Chafi, H., Wu, M., Atreya, A., Odersky, M., Olukotun, K.: OptiML: an implicitly parallel domain-specific language for machine learning. In: *Proceedings of the 28th International Conference on Machine Learning (ICML-11)*, pp. 609–616 (2011)
43. Wang, K., Sullivan, A., Marinov, D., Khurshid, S.: Asketch: a sketching framework for alloy. In: *Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (2018)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Juri Di Rocco is a postdoctoral researcher at the Department of Information Engineering Computer Science and Mathematics, Università degli Studi dell'Aquila, Italy. Previously, he obtained the PhD degree from Università degli Studi dell'Aquila in the MDE-Group research team with Alfonso Pierantonio and Davide Di Ruscio. He is interested in all aspects of software language engineering. Main research interests are related to several aspects of Model Driven Engineering (MDE) including

domain specific modelling languages, model transformation, model differencing, modelling repositories and mining techniques.



Davide Di Ruscio is an Associate Professor at the DISIM—University of L'Aquila. His main research interests are related to several aspects of Software Engineering, Open Source Software, and Model Driven Engineering (MDE) including domain specific modelling languages, model transformation, model differencing, coupled evolution, and recommendation systems. He has published more than 140 papers in various journals, conferences and workshops on such topics. He is a

member of the steering committee of the International Conference on Model Transformation (ICMT), of the Software Language Engineering (SLE) conference, of the Seminar Series on Advanced Techniques

and Tools for Software Evolution (SATTOSE), of the Workshop on Modelling in Software Engineering at ICSE (MiSE) and of the International Workshop on Robotics Software Engineering (RoSE). He is in the editorial board of the International Journal on Software and Systems Modeling (SoSyM), of IEEE Software, of the Journal of Object Technology, and of the IET Software journal. More information is available at <http://people.disim.univaq.it/diruscio>.



Claudio Di Sipio is a Ph.D. student at the University of L'Aquila, Italy. He is working on mining techniques to analyze open source software and he is also investigating the application of low-code platforms to support the development of recommendation systems.



Phuong T. Nguyen obtained a Ph.D. in Computer Science from the University of Jena, Germany. Since the graduation, he has worked as a university teaching and research assistant in Vietnam and Italy. Phuong is now with Università degli Studi dell'Aquila, Italy, as a postdoctoral researcher. His research interests include Computer Networks, Semantic Web, Recommender Systems, and Machine Learning. Recently, he has been working to develop recommender systems in Software

Engineering for mining open source code repositories.



Alfonso Pierantonio is Professor in Computer Science at Università degli Studi dell'Aquila, Italy. His current research interests include Model-Driven Engineering with a specific emphasis on model management analytics and co-evolution. Alfonso is involved in several scientific projects, including EU H2020 Lowcomote and EU H2020 Typhon; moreover, he is principal co-investigator of the Italian Railways funded project ERMES. He has been general chair of STAF 2015, PC chair of

ECMFA 2018, and designated general chair of MoDELS 2022 besides being on the organizing, steering, and program committees of many international conferences. Alfonso is the Editor-in-Chief of the Journal of Object Technology (JOT) and a member of the editorial board of the Journal on Software and Systems Modeling (SoSyM). He has published more than 140 papers in international journals, conferences, and workshops.